

PROPOSAL — THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG DATA LAKEHOUSE THEO LAMBDA ARCHITECTURE

Trường: Đại học Sư phạm Kỹ thuật TP.HCM (HCMUTE)

Chuyên ngành: Kỹ thuật Máy tính

Loại đề tài: Tiểu luận chuyên ngành (TLCN)

1. ĐẶT VẤN ĐỀ

1.1. Bối cảnh

Trong thời đại Big Data, các doanh nghiệp phải đối mặt với luồng dữ liệu liên tục từ nhiều nguồn: giao dịch tài chính, sự kiện người dùng, IoT sensors, log hệ thống. Hai nhu cầu đối lập thường xuyên xung đột nhau:

- **Nhu cầu phân tích lịch sử (Batch Analytics):** Cần xử lý hàng tỷ bản ghi với độ chính xác tuyệt đối, nhưng chấp nhận độ trễ (vài giờ/ngày).
- **Nhu cầu phản hồi thời gian thực (Real-time):** Cần dữ liệu cập nhật trong vài giây, nhưng chấp nhận kết quả gần đúng.

Kiến trúc truyền thống (Data Warehouse hoặc Data Lake đơn thuần) không thể đáp ứng đồng thời cả hai nhu cầu này.

1.2. Khoảng trống nghiên cứu

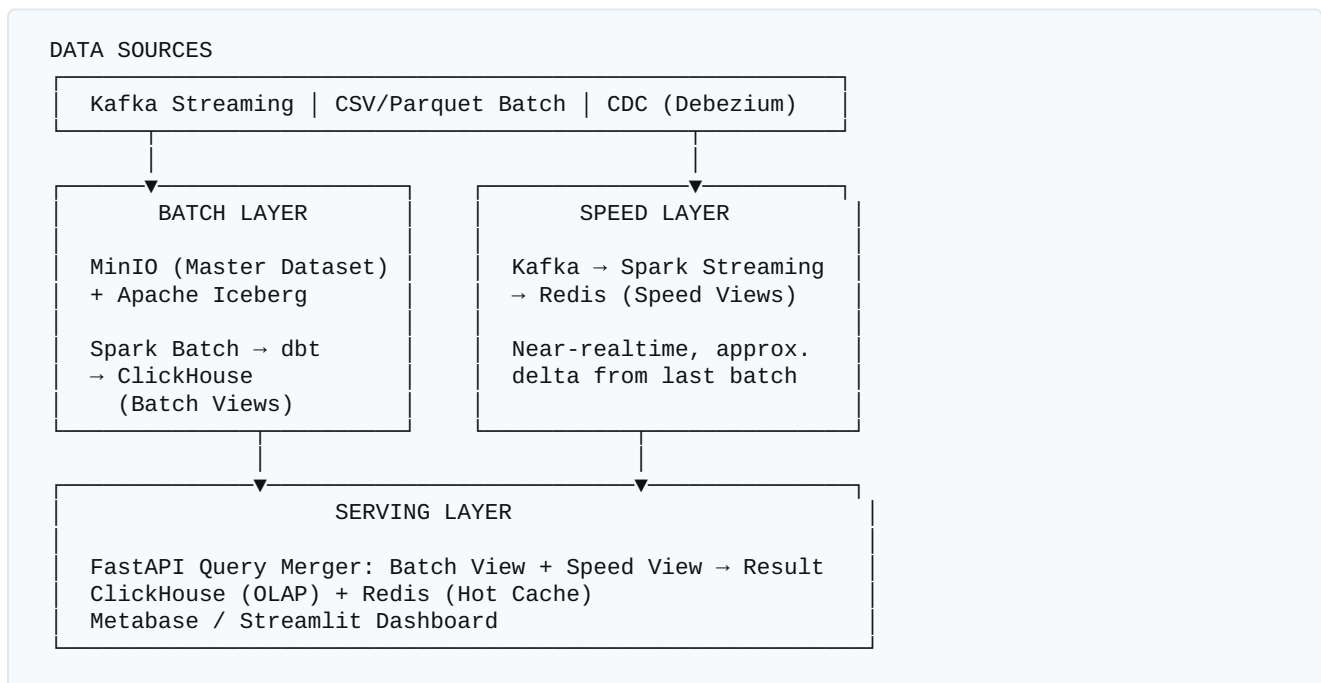
Vấn đề	Hệ thống truyền thống	Hướng giải quyết
Small File Problem	Streaming liên tục tạo hàng nghìn tệp nhỏ, làm suy giảm hiệu năng đọc	Iceberg Compaction (OPTIMIZE) theo lịch
Thiếu ACID Transactions	Ghi đồng thời gây bất nhất dữ liệu	Apache Iceberg với ACID guarantees
Không có Data Quality Gates	Dữ liệu lỗi lọt vào tầng phân tích	DQ constraints + Quarantine Tables
Batch vs Stream không tích hợp	Hai hệ thống riêng biệt, không có Serving Layer thống nhất	Lambda Architecture — hợp nhất qua Query Merger

2. MỤC TIÊU ĐỀ TÀI

1. **Thiết kế** hệ thống Data Lakehouse theo mô hình Lambda Architecture gồm 3 lớp: Batch Layer, Speed Layer và Serving Layer.
2. **Triển khai** pipeline hoàn chỉnh từ ingestion (Kafka) → Batch/Speed processing (Spark/Flink) → Serving (ClickHouse + Redis + FastAPI).

3. **Thực nghiệm & Đánh giá** hiệu năng qua 3 kịch bản benchmark cụ thể: latency, reprocessing correctness, và compaction optimization.
4. **So sánh** Lambda Architecture với các mô hình kiến trúc thay thế (Kappa, Medallion thuần).

3. LAMBDA ARCHITECTURE — TỔNG QUAN KIẾN TRÚC ĐỀ XUẤT



3.1. Batch Layer

- **Master Dataset:** MinIO + Apache Iceberg (immutable, bất biến)
- **Processing:** Apache Spark Batch Jobs, điều phối bởi Dagster
- **Transformation:** dbt models (Bronze → Silver → Gold)
- **Output:** Batch Views trong ClickHouse (cập nhật định kỳ: hourly/daily)

3.2. Speed Layer

- **Input:** Kafka topics (streaming events)
- **Processing:** Spark Structured Streaming hoặc Apache Flink
- **Output:** Speed Views trong Redis (key-value aggregations, TTL-based expiry)
- **Đặc điểm:** Approximate results, at-least-once semantics, low latency (<5s)

3.3. Serving Layer

- **Query Merger (FastAPI):** Tự động chọn Batch View (nếu đã có) + Speed View (delta bổ sung)
- **OLAP Engine:** ClickHouse — phục vụ ad-hoc analytics queries
- **Hot Cache:** Redis — phục vụ dashboard real-time
- **Orchestration:** Dagster quản lý toàn bộ pipeline lineage

4. KHUNG ĐỀ CƯƠNG

Chương 1: Tổng quan

- Bối cảnh Big Data và nhu cầu xử lý dữ liệu hỗn hợp (Batch + Stream)
- Phân tích hạn chế của các kiến trúc hiện tại (Data Warehouse, Data Lake, Medallion thuần)
- Mục tiêu, phạm vi, phương pháp nghiên cứu

Chương 2: Cơ sở Lý thuyết và Công nghệ

- **Lambda Architecture:** Lịch sử, 3 layer, ưu/nhược điểm
- **So sánh Lambda vs Kappa vs Medallion thuần**
- **Apache Iceberg:** Table Format, ACID, Time Travel, Partitioning
- **Kimball Dimensional Modeling:** Fact/Dimension, SCD Type 2
- **Data Quality:** Constraints, Quarantine, Metrics
- **Stack công nghệ:** Kafka, Spark, Flink, Redis, ClickHouse, Dagster, dbt

Chương 3: Thiết kế Kiến trúc Hệ thống

- Lựa chọn và mô tả nguồn dữ liệu
- Thiết kế Batch Layer: MinIO schema, Iceberg partition strategy, dbt models
- Thiết kế Speed Layer: Kafka topics, Spark/Flink jobs, Redis data structures
- Thiết kế Serving Layer: Query Merger logic, ClickHouse schema, FastAPI endpoints
- Orchestration với Dagster: Asset graph, schedules, data quality checks

Chương 4: Triển khai và Thực nghiệm

- Triển khai toàn bộ hệ thống bằng Docker Compose
- **Benchmark 1:** Latency comparison (Batch-only vs Speed-only vs Lambda Merge)
- **Benchmark 2:** Batch Reprocessing — Đo sai số Speed View vs Batch View sau khi batch hoàn tất
- **Benchmark 3:** Iceberg Compaction — Đo cải thiện hiệu năng sau khi gộp tệp nhỏ
- Đánh giá Data Quality: Tỷ lệ phát hiện lỗi, quarantine accuracy

Chương 5: Kết luận và Hướng phát triển

- Tổng kết kết quả thực nghiệm
- So sánh với kỳ vọng và giả thuyết ban đầu
- Hạn chế (logic viết 2 lần, vận hành phức tạp)
- Hướng phát triển: Tích hợp Feature Store / MLOps, chuyển sang Kappa nếu cần simplify

5. STACK CÔNG NGHỆ TỔNG HỢP

Phân hệ	Công nghệ	Ghi chú
Ingestion (Stream)	Apache Kafka + Debezium CDC	Real-time event bus
Ingestion (Batch)	Airbyte / Custom Spark Reader	Lịch sử + file dumps
Batch Processing	Apache Spark (PySpark)	Core compute engine
Stream Processing	Spark Structured Streaming	Speed Layer jobs
Transformation	dbt (Spark adapter)	Silver/Gold models
Storage	MinIO (S3-compatible)	Object store
Table Format	Apache Iceberg	ACID, Time Travel
Catalog	Iceberg REST Catalog	Schema registry
Batch Views	ClickHouse	OLAP engine
Speed Views	Redis	Hot cache, low-latency
Serving API	FastAPI	Query Merger + REST
Orchestration	Dagster	Asset lineage, schedules
Dashboard	Metabase / Streamlit	BI visualization
Containers	Docker Compose	Dev environment

6. ĐIỂM KHÁC BIỆT VÀ ĐÓNG GÓP HỌC THUẬT

1. **Triển khai Lambda Architecture end-to-end** với open-source stack trên môi trường local (Docker).
2. **Tích hợp Apache Iceberg** vào Master Dataset của Batch Layer — thay thế Parquet thô thông thường.
3. **Query Merger tự động** hợp nhất Batch và Speed Views mà không cần can thiệp thủ công.
4. **Benchmark định lượng** trên 3 chiều: latency, correctness, compaction efficiency.
5. **Có thể mở rộng** sang Feature Store / MLOps pipeline trong tương lai (đặt nền móng với Gold Layer và Redis Feature Cache).